Rapport P01

Capture et analyse d'un trafic réseau d'entreprise simulée

ESTI – Licence 2 Informatique – Sécurité des Réseaux · Projet de groupe P01 (chapitre 1) · Rapport du 29/10/2026 · Remise le 30/10/2026

Membres du groupe : Miriah, Joe, Soatiana, Laurie

Dépôt Git : https://github.com/Miriah-santatriniaina/p01-capture-analyse-trafic

1. Contexte et objectifs

Ce projet montre, dans un laboratoire isolé, ce qu'un attaquant en position de sniffing lit sur un réseau d'entreprise simulé, puis ce qui change quand la même communication est chiffrée. Il correspond au sujet P01 du catalogue (chapitre 1 du cours).

Le sujet demande de monter un petit réseau (client, serveur web, DNS), de capturer le trafic avec Wireshark ou tcpdump, d'identifier les protocoles en clair (HTTP, FTP, Telnet, DNS), d'en extraire des identifiants, puis de refaire la même communication en HTTPS et SSH pour comparer. Il impose aussi une phase offensive (ou de test) et une phase défensive (ou de correction).

Objectifs du projet

- Observer en pratique la fuite d'informations d'un trafic non chiffré : identifiants, données transférées, noms consultés.
- Extraire ces informations avec des outils standard (`tcpdump`, `tshark`, Wireshark), sans aucune attaque active.
- Rejouer les mêmes échanges en HTTPS et SSH et mesurer ce que l'attaquant ne voit plus.
- Corriger l'architecture : ne laisser passer que les protocoles chiffrés et prouver le blocage des autres.

Cadre éthique et légal. Toutes les captures ont été réalisées dans un laboratoire appartenant au groupe, sans exposition à Internet et sans cibler un réseau tiers. Les comptes utilisés (`labuser`, `webadmin`) ont été créés uniquement pour le projet, avec des mots de passe volontairement simples. Le trafic capturé ne concerne que les trois machines virtuelles du laboratoire.

Correspondance avec les exigences du sujet

Exigence du sujet	Où la trouver dans ce rapport
Architecture avec schéma	Section 3
Étapes de mise en œuvre	Section 4
Phase offensive : protocoles en clair et identifiants	Section 5
Même communication en HTTPS et SSH, comparaison	Section 6
Phase défensive (correction)	Section 7
Analyse de sécurité, limites, recommandations	Sections 8 et 9
Éléments reproductibles (scripts, dépôt Git)	Annexes et dépôt Git
Preuves (captures d'écran, journaux, fichiers pcap)	Sections 4 à 7
Rôle de chaque membre	Section 10

2. Environnement technique et justification

Le laboratoire repose sur trois machines virtuelles Ubuntu 20.04.6 LTS sous VMware Workstation, hébergées sur un PC Windows. Ce choix réutilise l'infrastructure du laboratoire réseau INFO 351 (DHCP, DNS, routage, NAT, iptables), déjà validée, ce qui laisse le temps du projet à la capture et à l'analyse.

Justification du choix des machines virtuelles

- **Isolation :** les réseaux LAN 1 et LAN 2 sont des segments virtuels sans lien avec le réseau physique ; seul vm1 dispose d'une carte NAT, utilisée pour installer des paquets.
- **Réalisme :** vm1 est un vrai routeur Linux entre deux réseaux, avec iptables. Un conteneur ne reproduit pas aussi bien la traversée d'un routeur ni le filtrage de transit (chaîne FORWARD).
- **Réversibilité :** un snapshot (`avant-P01`) a été pris avant toute modification, pour revenir à l'état initial.
- **Contrainte de temps :** Docker ou Kubernetes auraient demandé de reconstruire le réseau, sans bénéfice pour le sujet.

Logiciels utilisés

Fonction	Logiciel	Version relevée
Serveur web, HTTP et HTTPS	Apache	2.4.41 (Ubuntu)
Serveur FTP	vsftpd	3.0.5 (bannière 220 visible dans la capture)
Serveur Telnet	telnetd lancé par openbsd-inetd	paquet Ubuntu 20.04
SSH	OpenSSH	8.2p1 (Ubuntu 4ubuntu0.13)
Résolution de noms	BIND9, zone `esti.mg`	laboratoire INFO 351
Client de test	curl, ftp, telnet, ssh	curl 7.68.0
Capture et analyse	tcpdump (vm1), tshark (vm1), Wireshark (poste Windows)	tshark 3.2.3
Pare-feu	iptables	chaîne dédiée P01

3. Architecture

Le laboratoire compte trois machines et deux réseaux : le client vm2 (LAN 1) et le serveur vm3 (LAN 2) ne communiquent que par le routeur vm1, qui est donc l’endroit idéal pour observer le trafic.

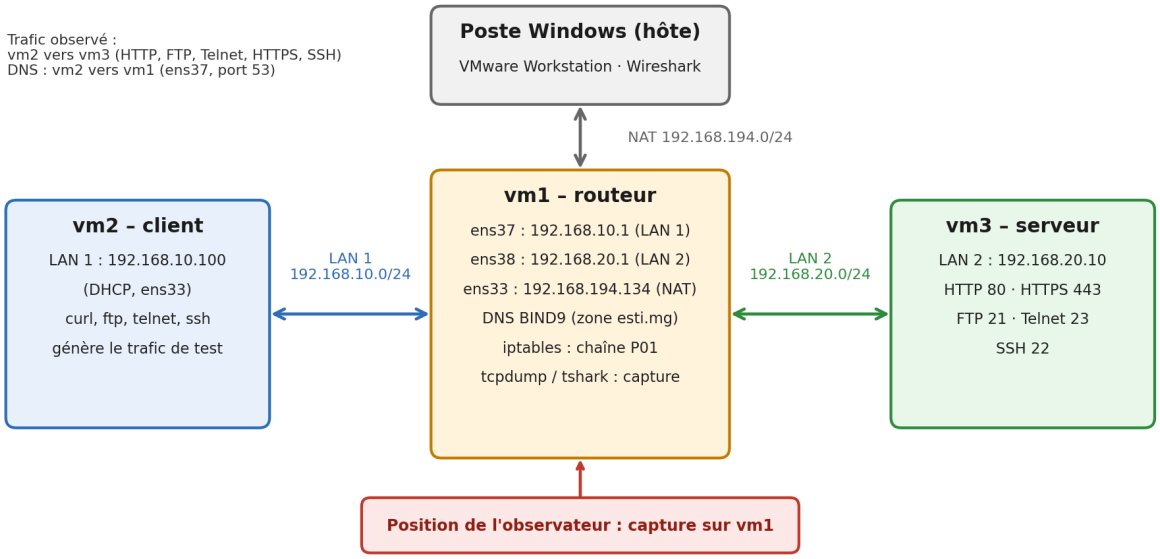


Schéma 1 : architecture du laboratoire P01 (réutilisée du laboratoire INFO 351).

Rôle des machines

Machine	Rôle dans P01	Interfaces et adresses
vm1	Routeur, serveur DNS (BIND9), pare-feu iptables, point de capture	`ens33` 192.168.194.134 (NAT) · `ens37` 192.168.10.1 (LAN 1) · `ens38` 192.168.20.1 (LAN 2)
vm2	Client, source du trafic légitime	`ens33` 192.168.10.100 (DHCP, LAN 1)
vm3	Serveur : Apache, FTP, Telnet, SSH	`ens33` 192.168.20.10 (statique, LAN 2)

Plan d'adressage

Réseau	Plage	Passerelle	Machine
LAN 1	192.168.10.0/24	192.168.10.1 (vm1, ens37)	vm2 : 192.168.10.100 (DHCP, plage .100 à .200)
LAN 2	192.168.20.0/24	192.168.20.1 (vm1, ens38)	vm3 : 192.168.20.10 (statique)
NAT VMware	192.168.194.0/24	fournie par VMware	vm1 : 192.168.194.134 (ens33, DHCP)

Noms DNS de la zone esti.mg (BIND9 sur vm1)

Nom	Adresse	Usage dans P01
`web.esti.mg`	192.168.20.10	serveur vm3, cible de tous les tests
`www.esti.mg`, `proxy.esti.mg`	192.168.194.134	noms hérités du laboratoire INFO 351, utilisés pour les requêtes DNS capturées

Flux étudiés

Protocole	Port	Chemin	Interface de capture sur vm1
HTTP, HTTPS	80, 443	vm2 vers vm3	`ens38`
FTP	21 et 40000 à 40010 (mode passif)	vm2 vers vm3	`ens38`
Telnet	23	vm2 vers vm3	`ens38`
SSH	22	vm2 vers vm3	`ens38`

Protocole	Port	Chemin	Interface de capture sur vm1
DNS	53 (UDP)	vm2 vers vm1	`ens37`

Isolation. Les réseaux LAN 1 et LAN 2 sont des segments virtuels VMware sans lien avec le réseau physique. Aucune des machines n'est exposée à Internet : la carte NAT de vm1 sert uniquement à installer des paquets, et toutes les attaques et captures restent à l'intérieur du laboratoire.

4. Mise en œuvre

La mise en place tient en quatre étapes : préparer le serveur vm3, ouvrir les flux sur le routeur vm1, vérifier la connectivité depuis le client vm2, puis capturer. Les deux scripts utilisés (`p01_vm3_install.sh` et `p01_vm1_flux.sh`) sont fournis dans le dépôt Git.

4.1 Préparation du serveur vm3

Le script `p01_vm3_install.sh` installe et configure, volontairement sans chiffrement pour les trois premiers services :

- **FTP :** vsftpd, accès anonyme désactivé, comptes locaux autorisés, ports passifs 40000 à 40010 (pour pouvoir les filtrer précisément).
- **Telnet :** telnetd lancé par openbsd-inetd sur le port 23.
- **HTTP avec authentification :** zone `/prive` d'Apache protégée par authentification basique (`webadmin`, fichier `.htpasswd`).
- **HTTPS :** module ssl et site `p01-ssl` sur le port 443, avec un certificat auto-signé RSA 2048 bits (CN `web.esti.mg`, 365 jours).
- **SSH :** OpenSSH, déjà présent et activé.
- **Compte de test** `labuser` utilisé par Telnet, FTP et SSH, avec un fichier `confidentiel.txt` de 43 octets (`Fichier de test P01 - contenu confidentiel`).

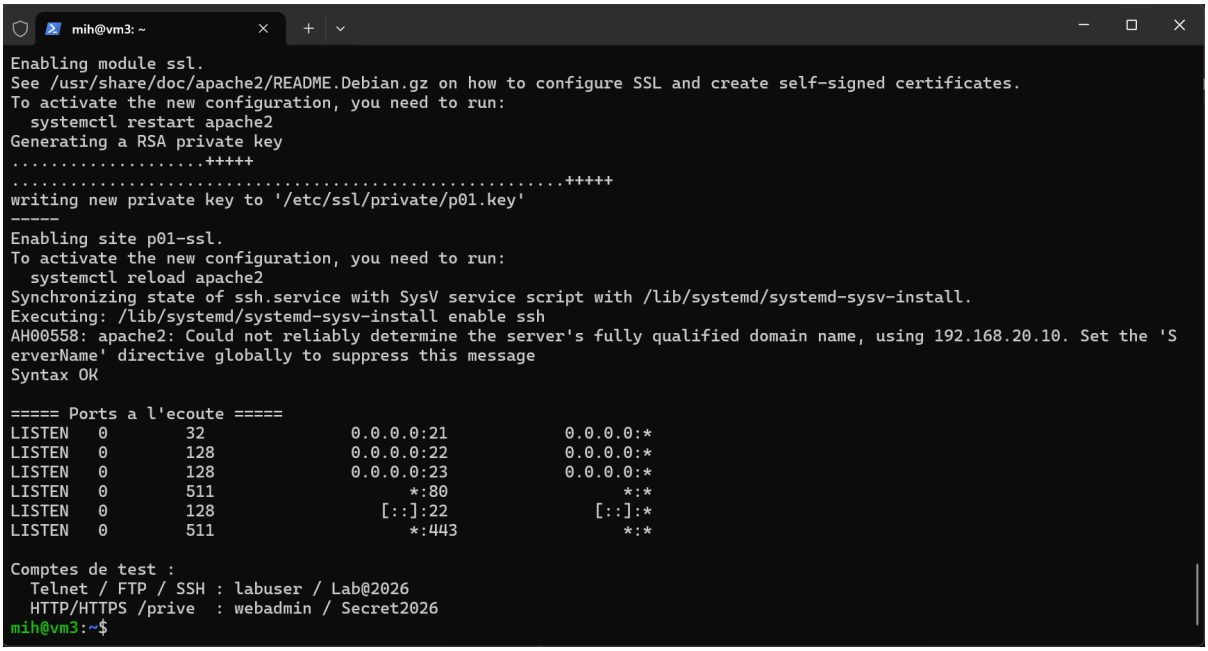


Figure 1 : fin du script d'installation sur vm3. Apache répond `Syntax OK` et les ports 21, 22, 23, 80 et 443 sont à l'écoute (l'avertissement AH00558 sur le nom du serveur est sans conséquence).

4.2 Ouverture des flux sur le routeur vm1

Depuis le laboratoire INFO 351, la chaîne FORWARD de vm1 est en politique `DROP` et refuse tout flux direct du LAN 1 vers le LAN 2. Pour pouvoir observer le trafic, le script `p01_vm1_flux.sh offensif` crée une chaîne dédiée `P01`, insérée en tête de FORWARD. Elle contient deux règles : le retour des réponses (`RELATED,ESTABLISHED`) et l'autorisation de 192.168.10.0/24 vers 192.168.20.10 sur les ports 21, 22, 23, 80, 443 et 40000 à 40010. La commande `reset` supprime la chaîne et restaure l'état initial.

```
mih@vm1:~$ wc -l ~/p01_vm1_flux.sh
52 /home/mih/p01_vm1_flux.sh
mih@vm1:~$ sudo bash ~/p01_vm1_flux.sh offensif
[sudo] password for mih:
Phase offensive : flux en clair ET chiffres ouverts vers 192.168.20.10
Chain P01 (1 references)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     all  --  ens38  ens37   0.0.0.0/0            0.0.0.0/0            state RELATED,ESTABLISHED
2        0     0 ACCEPT     tcp  --  ens37  ens38   192.168.10.0/24      192.168.20.10        multiport dports 21,22,23,
80,443,40000:40010
mih@vm1:~$
```

Figure 2 : chaîne `P01` en phase offensive, avec la règle de retour et la règle d'autorisation vers vm3.

4.3 Vérification depuis le client vm2

Cinq tests confirment que chaque service répond avant de commencer à capturer : résolution de `web.esti.mg` (192.168.20.10), page `/prive/` avec identifiants, liste du répertoire FTP (on y voit `confidentiel.txt`), HTTPS (`code 200`) et port 23 ouvert.

```
Reading state information... Done
ftp is already the newest version (0.17-34.1).
ftp set to manually installed.
telnet is already the newest version (0.17-41.2build1).
telnet set to manually installed.
curl is already the newest version (7.68.0-1ubuntu2.25).
0 upgraded, 0 newly installed, 0 to remove and 106 not upgraded.
mih@vm2:~$ getent hosts web.esti.mg
192.168.20.10   web.esti.mg
mih@vm2:~$ curl -u webadmin:Secret2026 http://web.esti.mg/prive/
<html><body><h1>Zone privee - Projet P01</h1>
<p>Si vous lisez ceci, l'authentification a reussi.</p></body></html>
mih@vm2:~$ curl -u labuser:Lab@2026 ftp://web.esti.mg/
-rw-r--r--    1 1001     1001           43 Oct 27 06:00 confidentiel.txt
mih@vm2:~$ curl -k https://web.esti.mg/ -o /dev/null -w "HTTPS : code %{http_code}\n"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   381  100   381    0     0   6145      0 --:--:-- --:--:-- --:--:--  6245
HTTPS : code 200
mih@vm2:~$ timeout 3 bash -c '</dev/tcp/web.esti.mg/23' && echo "Telnet : port 23 ouvert"
Telnet : port 23 ouvert
mih@vm2:~$
```

Figure 3 : les cinq tests depuis vm2 réussissent.

4.4 Méthode de capture

vm1 est le routeur entre les deux réseaux : tout le trafic entre vm2 et vm3 la traverse. La capture se fait donc avec `tcpdump -i ens38 -nn -s 0 -w fichier.pcap` et un filtre sur l'hôte 192.168.20.10 et le port du protocole étudié, côté LAN 2. Seul le DNS est capturé sur `ens37` (port 53), car vm2 interroge vm1 en 192.168.10.1. Les fichiers sont ensuite lus sur place avec `tshark`, puis copiés sur le poste Windows par `scp` pour des captures d'écran dans Wireshark.

Fichier	Protocole	Filtre de capture	Paquets
`clair_http.pcap`	HTTP, authentification basique	`host 192.168.20.10 and port 80`	10
`clair_ftp.pcap`	FTP	`host 192.168.20.10 and (port 21 or portrange 40000-40010)`	45
`clair_telnet.pcap`	Telnet	`host 192.168.20.10 and port 23`	90
`clair_dns.pcap`	DNS	`port 53 and host 192.168.10.100` (sur ens37)	26

Fichier	Protocole	Filtre de capture	Paquets
`chiffre_https.pcap`	HTTPS	`host 192.168.20.10 and port 443`	26
`chiffre_ssh.pcap`	SSH	`host 192.168.20.10 and port 22`	52

Cette position équivaut à celle d'un attaquant qui a compromis le routeur, ou qui reçoit une copie du trafic sur un port miroir d'un commutateur. Elle ne nécessite aucune attaque active (pas d'usurpation ARP, qui fait l'objet d'un autre sujet du catalogue).

5. Phase offensive : ce que lit l'attaquant en clair

Sur les quatre protocoles en clair testés, un attaquant qui observe le trafic récupère identifiants, mots de passe et contenus de fichiers sans aucune attaque active, avec deux commandes (`tcpdump` puis `tshark`).

Protocole	Port	Ce qui est lisible dans la capture	Résultat
HTTP (auth. basique)	80	en-tête `Authorization`, décodé en `webadmin:Secret2026`	identifiant et mot de passe
FTP	21 + ports passifs	`USER labuser`, `PASS Lab@2026`, contenu du fichier transféré	identifiant, mot de passe, données
Telnet	23	tout l'échange interactif, mot de passe et commandes compris	session complète
DNS	53 (UDP)	noms demandés et adresses renvoyées	cartographie du réseau

5.1 HTTP avec authentification basique

Scénario. Depuis vm2 : `curl -u webadmin:Secret2026 http://web.esti.mg/prive/`. La requête réussit et affiche la page « Zone privée ».

Ce que voit l'attaquant. La requête `GET /prive/` porte l'en-tête `Authorization: Basic d2ViYWRtaW46U2VjcmV0MjAyNg==`. Cette chaîne est le **base64** de `webadmin:Secret2026` : un simple encodage réversible, pas un chiffrement. Wireshark le décode lui-même (`Credentials: webadmin:Secret2026`). La commande `tshark -r clair_http.pcap -Y http.authbasic -T fields -e ip.src -e http.authbasic` retourne `192.168.10.100 webadmin:Secret2026`.

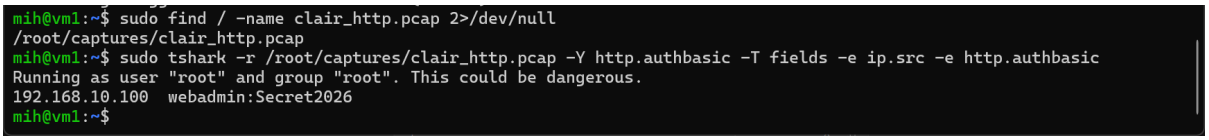


Figure 4 : extraction en une commande avec tshark sur vm1.

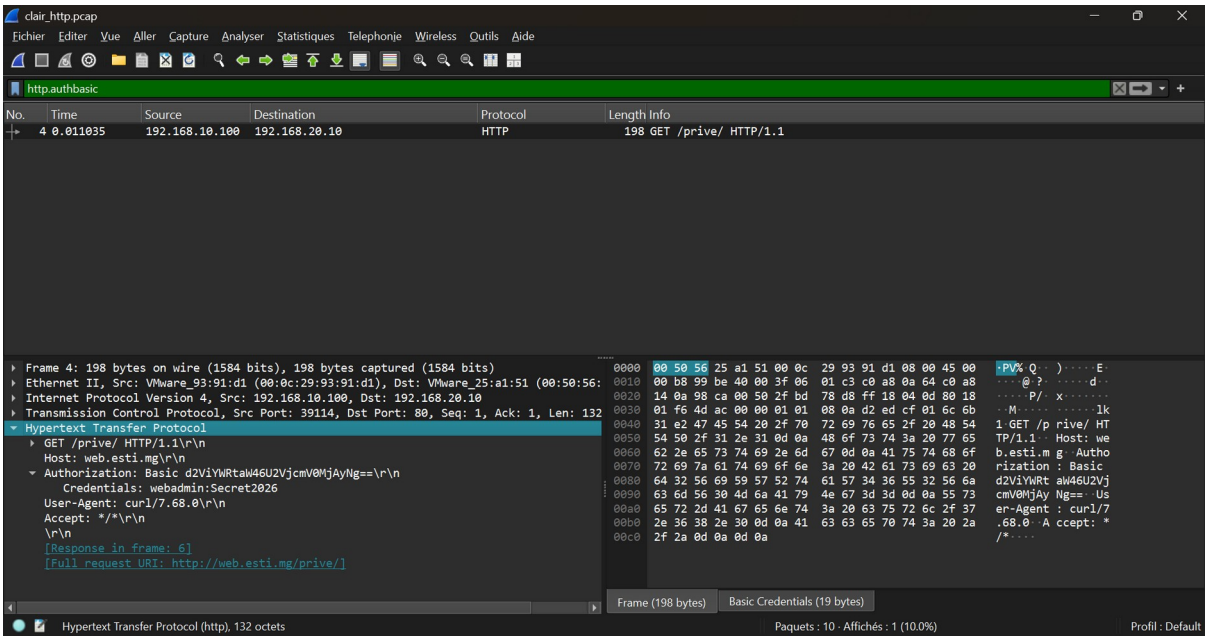


Figure 5 : dans Wireshark, filtre `http.authbasic` : l'en-tête Authorization et les identifiants décodés sont visibles.

5.2 FTP

Scénario. Depuis vm2 : `curl -u labuser:Lab@2026 ftp://web.esti.mg/confidentiel.txt`. Le contenu du fichier s'affiche.

Ce que voit l'attaquant. Le canal de commandes (port 21) montre `Response: 220 (vsFTPd 3.0.5)`, `USER labuser`, `331 Please specify the password.`, `PASS Lab@2026` puis `230 Login successful.`. Le canal de données (port passif) transporte le fichier : la capture contient les commandes `SIZE` et `RETR confidentiel.txt`, la réponse `150 Opening BINARY mode data connection for confidentiel.txt (43 bytes)` et le texte `Fichier de test P01 - contenu confidentiel`. L'attaquant obtient donc le mot de passe et les données transférées.

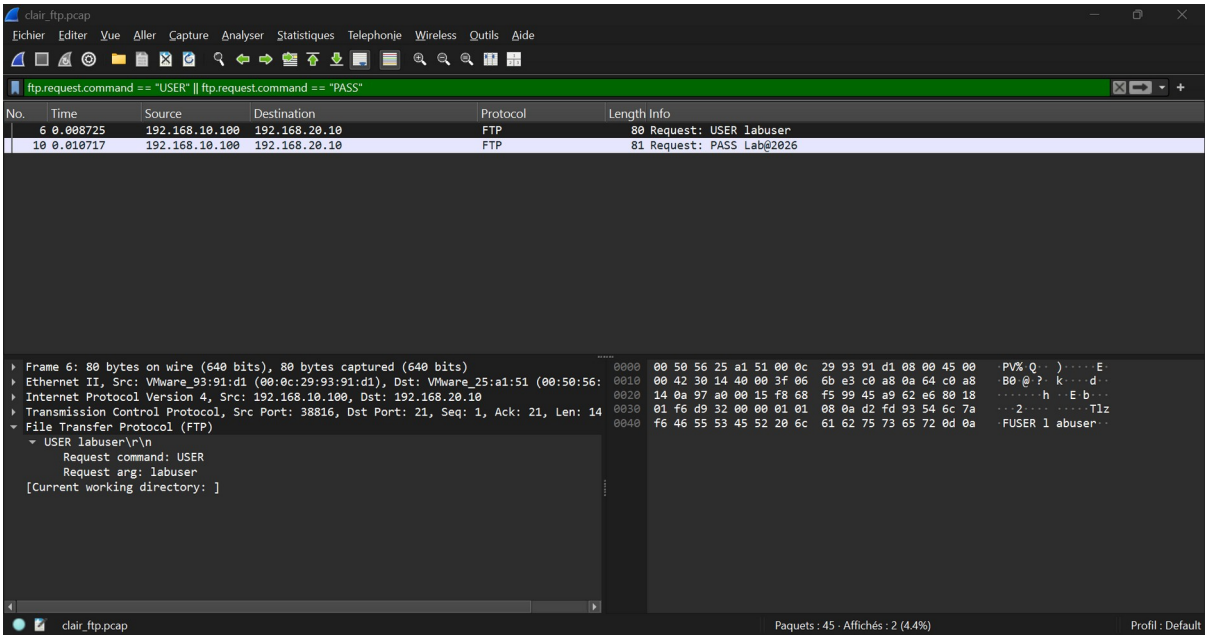


Figure 6 : filtre `ftp.request.command == "USER" || ftp.request.command == "PASS"` : les deux commandes sont lisibles.

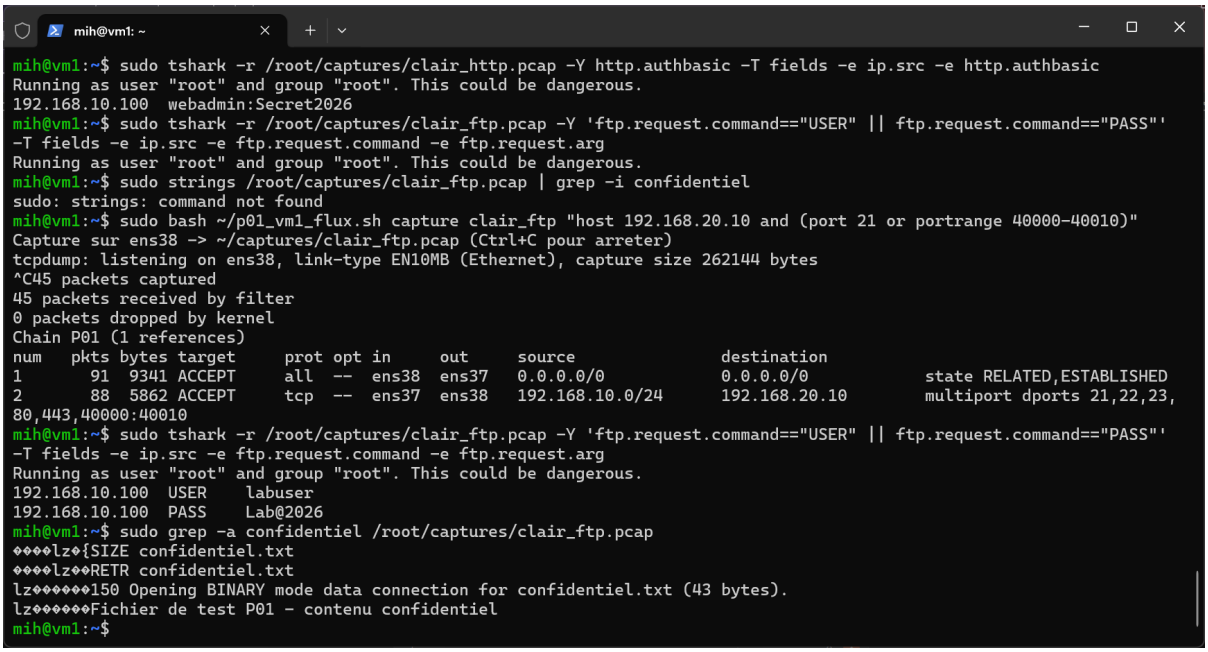


Figure 7 : `tshark` affiche `USER labuser` et `PASS Lab@2026` ; `grep -a confidentiel` retrouve le contenu du fichier dans la capture (45 paquets).

5.3 Telnet

Scénario. Depuis vm2 : `telnet web.esti.mg`, connexion avec `labuser`, puis `whoami`, `cat confidentiel.txt` et `exit`.

Ce que voit l'attaquant. La fonction « Suivre le flux TCP » de Wireshark reconstitue toute la session (1699 octets, 21 paquets du client, 27 du serveur) : l'invite `vm3 login:`, l'identifiant `labuser`, l'invite `Password:`, le mot de passe `Lab@2026`, puis les commandes tapées et leurs résultats, dont le contenu de `confidentiel.txt`. Les lignes en double sont l'écho que le serveur renvoie pour chaque caractère. On voit aussi que `exit` part lettre par lettre, un paquet par touche : Telnet expose même les frappes.

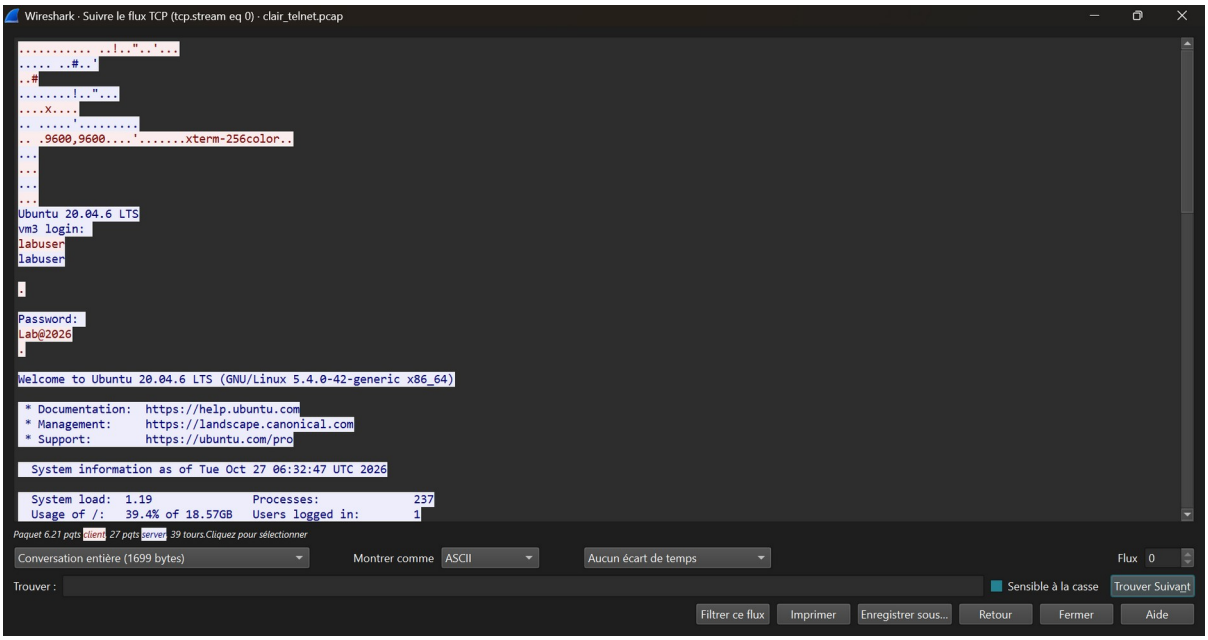


Figure 8 : flux TCP de la session Telnet, identifiant et mot de passe en clair (rouge : client, bleu : serveur).

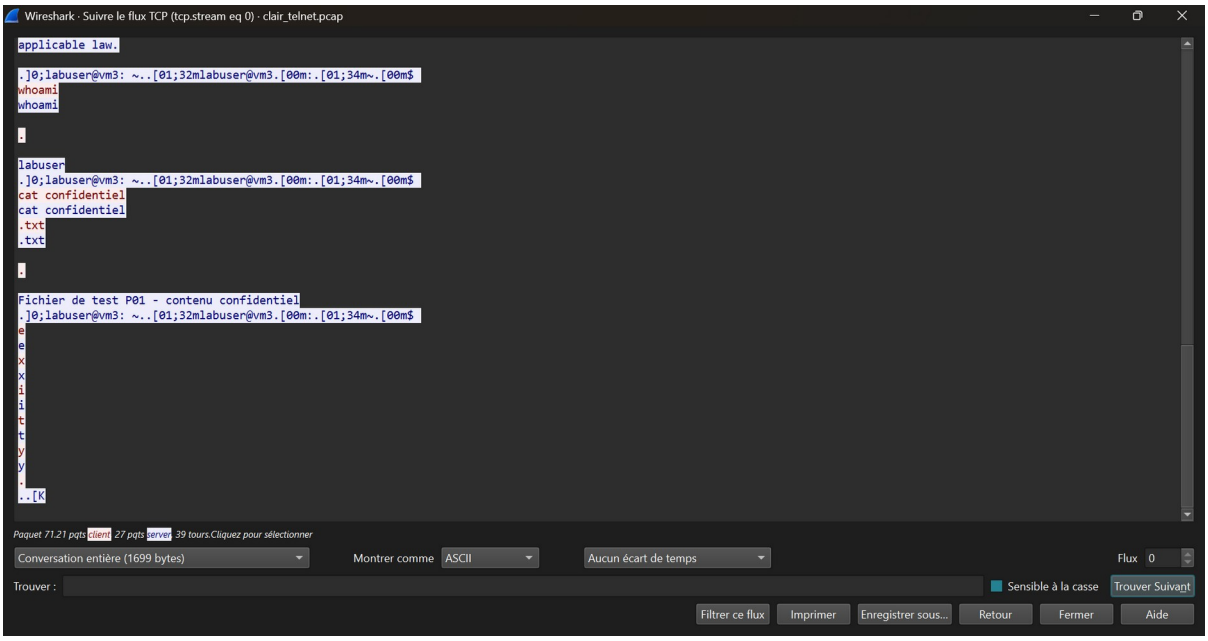


Figure 9 : suite de la session, avec `whoami`, `cat confidentiel.txt`, son contenu et `exit` envoyé caractère par caractère.

5.4 DNS

Scénario. Après avoir vidé le cache de vm2, `getent hosts web.esti.mg www.esti.mg proxy.esti.mg google.com`. La capture se fait sur l'interface `ens37` de vm1.

Ce que voit l'attaquant. Le DNS circule en UDP port 53 sans aucune protection. La capture (26 paquets) révèle les noms consultés par vm2 et les réponses du serveur : `web.esti.mg` pour 192.168.20.10, `www.esti.mg` et `proxy.esti.mg` pour 192.168.194.134, et la consultation de `google.com`. Le client demande d'abord l'adresse IPv6 (type `AAAA`) puis IPv4 (type `A`). Les requêtes `web.esti.mg.esti.mg`, qui reçoivent `No such name`, viennent du suffixe de recherche `esti.mg` ajouté par vm2 : sans danger, mais visibles. Même quand le contenu est chiffré, l'attaquant sait donc quels serveurs sont contactés et à quelles adresses.

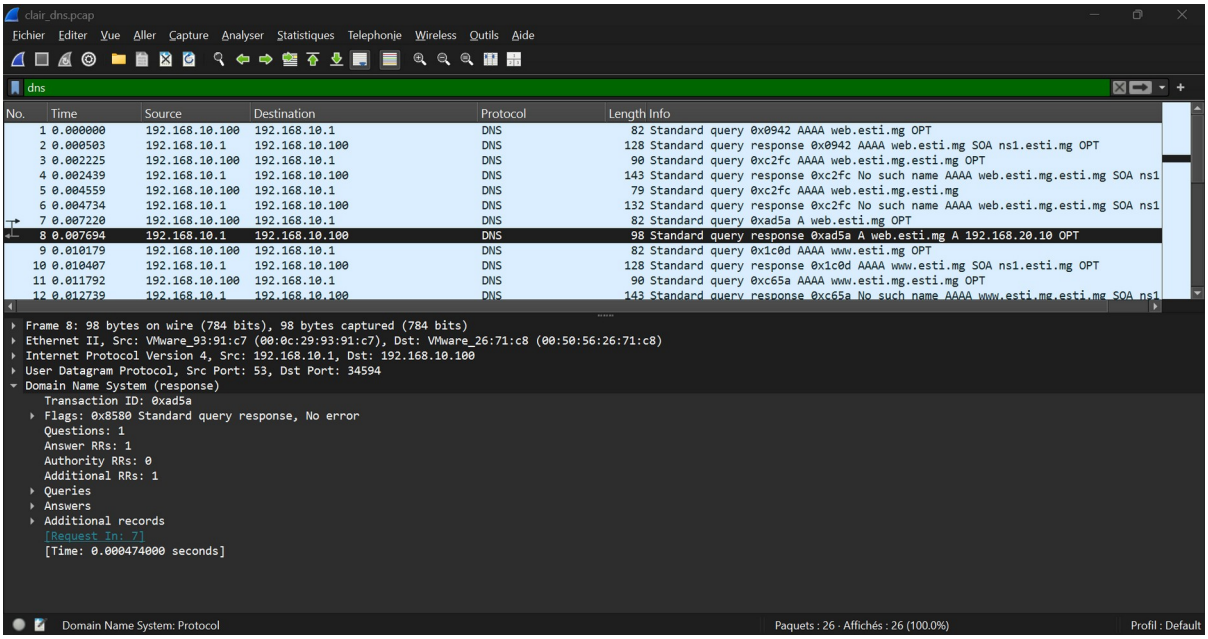


Figure 10 : filtre `dns` ; la réponse n° 8 associe `web.esti.mg` à 192.168.20.10.

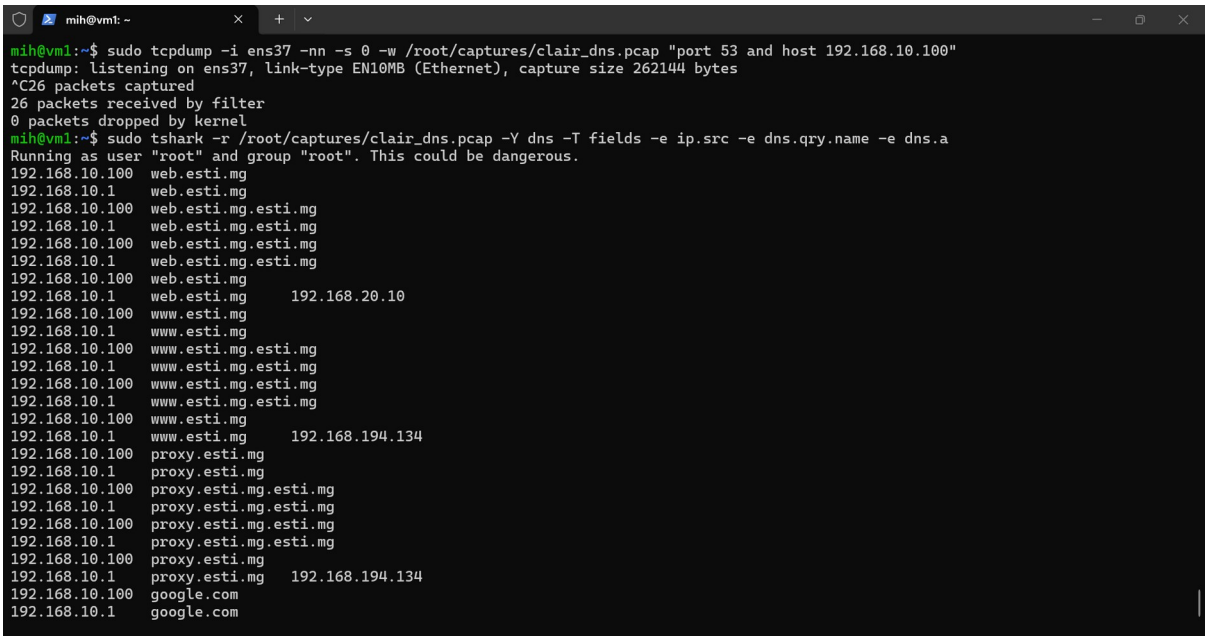


Figure 11 : `tshark -Y dns` liste les noms demandés par 192.168.10.100 et les adresses renvoyées par 192.168.10.1.

6. Même communication en HTTPS et SSH

Les mêmes échanges, refaits en HTTPS et en SSH, donnent le même résultat côté utilisateur, mais la capture ne contient plus ni mot de passe ni contenu : seules restent la négociation du chiffrement et des métadonnées.

6.1 HTTPS

Scénario. Depuis vm2 : `curl -k -u webadmin:Secret2026 https://web.esti.mg/prive/`. Les identifiants et la page sont les mêmes qu'en HTTP ; l'option `-k` accepte le certificat auto-signé du laboratoire.

Preuves. Sur la capture `chiffre_https.pcap` (26 paquets), le filtre `http.authbasic` ne retourne rien, `grep -a -c Secret2026` retourne `0`, et seuls apparaissent un `Client Hello` (paquet 4) puis un `Server Hello, Change Cipher Spec` suivi d'`Application Data` (paquet 6).

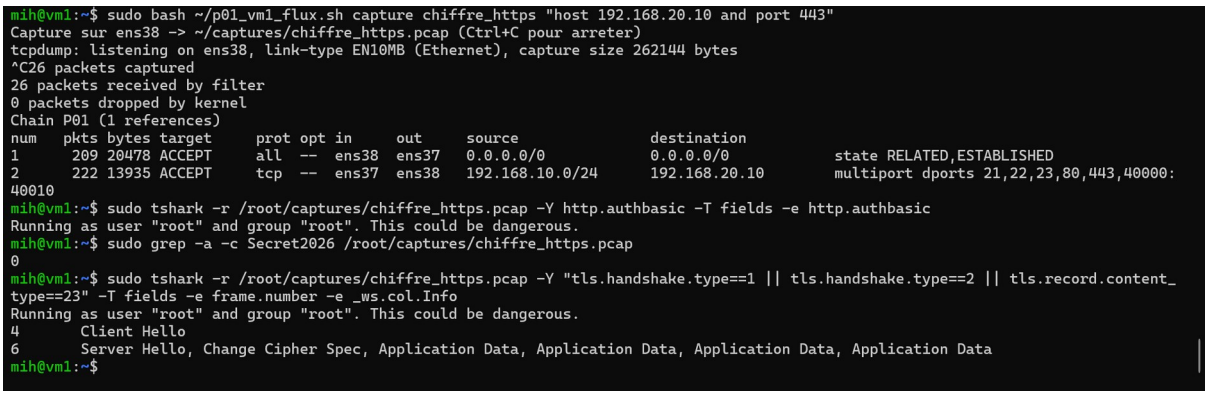


Figure 12 : sur vm1, aucun champ d'authentification, aucune occurrence du mot de passe, uniquement l'ouverture de la session TLS.

Analyse du handshake. Wireshark indique `TLSv1.3` dans la colonne Protocole. Le champ `Version: TLS 1.2 (0x0303)` du Server Hello est une valeur de compatibilité conservée par TLS 1.3 ; la version réellement négociée se lit dans l'extension `supported_versions` : `TLS 1.3`. La suite de chiffrement est `TLS_AES_256_GCM_SHA384` et l'échange de clés utilise la courbe `x25519` (extension `key_share`). Dès le

Server Hello, le reste du handshake est chiffré (`Application Data`). Une information reste cependant lisible : le `Client Hello` contient `SNI=web.esti.mg`, le nom du serveur visé.

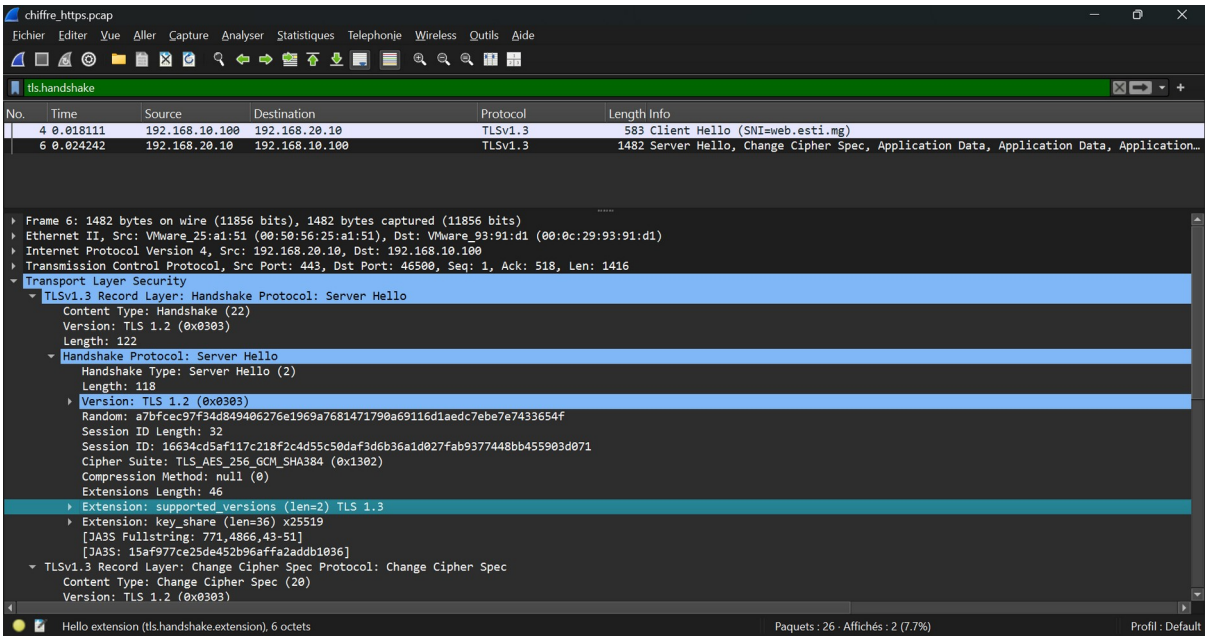


Figure 13 : filtre `tls.handshake`, Server Hello déplié : extension `supported_versions` (TLS 1.3), suite `TLS_AES_256_GCM_SHA384`, `key_share` x25519.

6.2 SSH

Scénario. Depuis vm2 : `ssh labuser@web.esti.mg 'whoami; cat confidentiel.txt'`, avec saisie du mot de passe. Le résultat s'affiche normalement.

Preuves. Sur `chiffre_ssh.pcap` (52 paquets), `grep -a -c -e Lab@2026 -e confidentiel` retourne `0`. Seuls restent lisibles les bannières (`SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.13`), l'échange de clés Diffie-Hellman sur courbe elliptique (`curve25519-sha256`) et `New Keys`. À partir de là, tous les paquets sont des `Encrypted packet`, y compris la saisie du mot de passe et le contenu du fichier. Le chiffrement négocié est `chacha20-poly1305@openssh.com`.

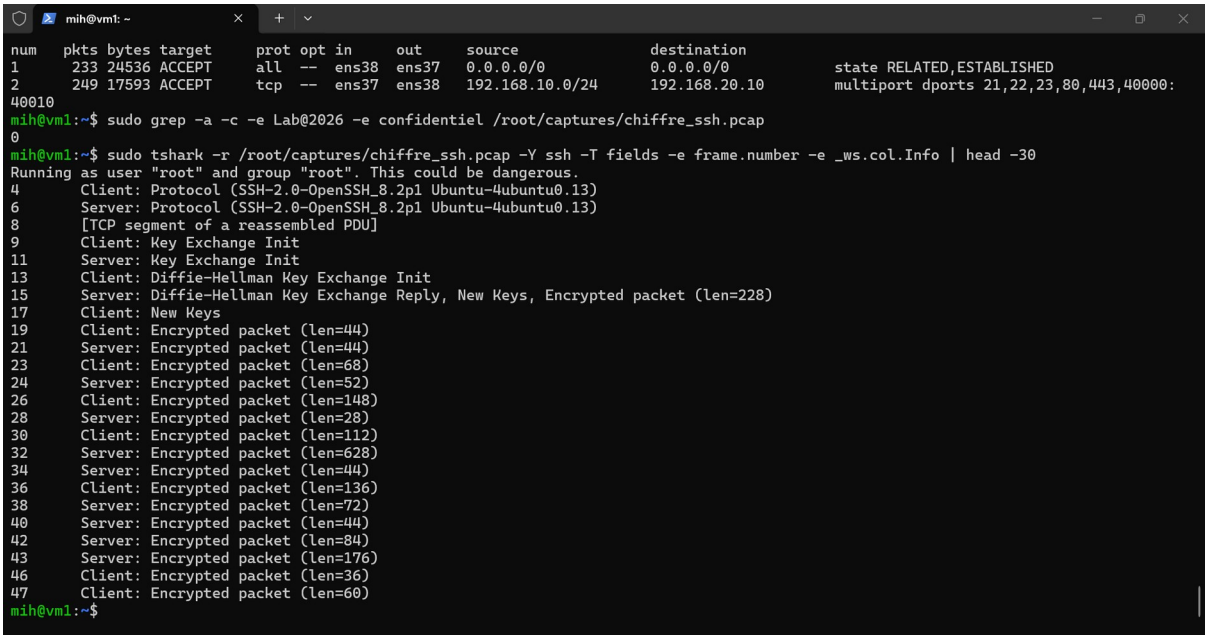


Figure 14 : aucune occurrence du mot de passe ni du fichier ; la liste des paquets SSH passe de l'échange de clés aux `Encrypted packet`.

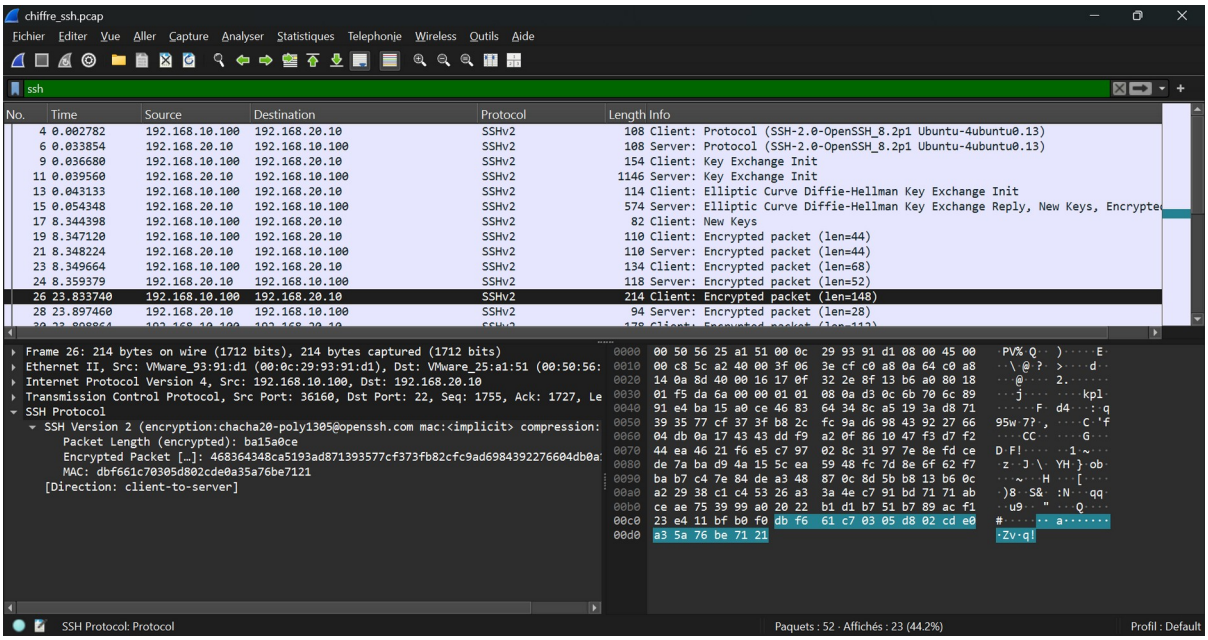


Figure 15 : un paquet chiffré du client vers le serveur (len=148) : longueur chiffrée, suite d'octets illisibles et MAC, sans aucun texte clair.

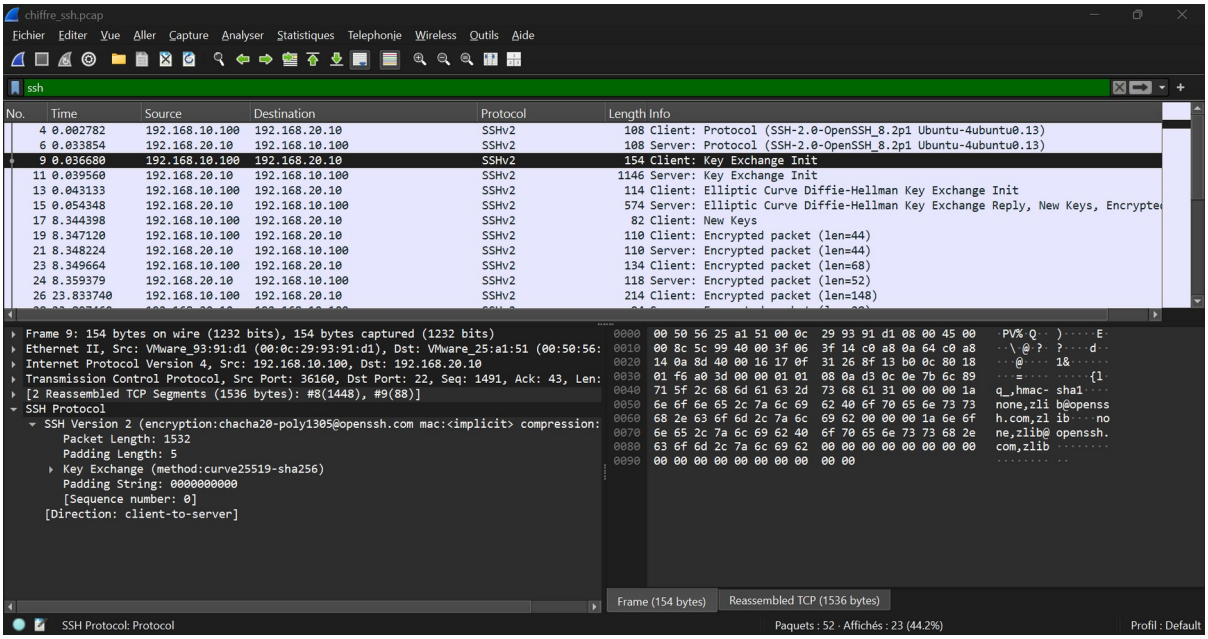


Figure 16 : le `Key Exchange Init` (paquet 9) montre que les algorithmes sont négociés avant tout échange de données.

6.3 Bilan de la comparaison

En clair	Équivalent chiffré	Identifiants	Contenu	Ce qui reste visible malgré le chiffrement
HTTP (80)	HTTPS (443)	lisibles (`webadmin:Secret2026`) → introuvables	page et en-têtes → `Application Data`	adresses IP, ports, nom `web.esti.mg` (SNI), tailles et horaires
Telnet (23)	SSH (22)	`labuser` et `Lab@2026` lisibles → introuvables	commandes et fichier → `Encrypted packet`	adresses IP, ports, version OpenSSH, algorithmes, tailles des paquets
FTP (21)	SFTP ou FTPS (non testés)	lisibles	fichier lisible	à tester

En clair	Équivalent chiffré	Identifiants	Contenu	Ce qui reste visible malgré le chiffrement
DNS (53, UDP)	DNS sur TLS ou HTTPS (non testés)	sans objet	noms et adresses lisibles	à tester

7. Phase défensive

Après la phase offensive, le pare-feu de vm1 ne laisse plus passer vers vm3 que SSH (port 22) et HTTPS (port 443), et journalise les tentatives sur FTP, Telnet et HTTP.

7.1 Correction appliquée

La commande `sudo bash ~/p01_vm1_flux.sh defensif` recrée la chaîne `P01` avec trois règles :

1. le retour des réponses (`RELATED,ESTABLISHED`) de LAN 2 vers LAN 1 ;
2. `ACCEPT` de 192.168.10.0/24 vers 192.168.20.10 sur les ports `22,443` uniquement ;
3. `LOG` des paquets vers les ports `21,23,80`, avec le préfixe `P01-BLOCK`.

Tout paquet non accepté tombe sur la politique `DROP` de la chaîne FORWARD. Les ports passifs FTP (40000 à 40010) sont aussi refermés.

7.2 Tests de blocage et de bon fonctionnement

Test depuis vm2	Attendu	Obtenu
`curl --max-time 5 http://web.esti.mg/prive/` (HTTP)	bloqué	`Connection timed out`, code retour 28
`curl --max-time 5 ftp://web.esti.mg/` (FTP)	bloqué	`Connection timed out`, code retour 28
connexion au port 23 (Telnet)	bloqué	`Telnet : bloque`
`curl -k https://web.esti.mg/` (HTTPS)	autorisé	`HTTPS : code 200`
connexion au port 22 (SSH)	autorisé	`SSH : port 22 ouvert`

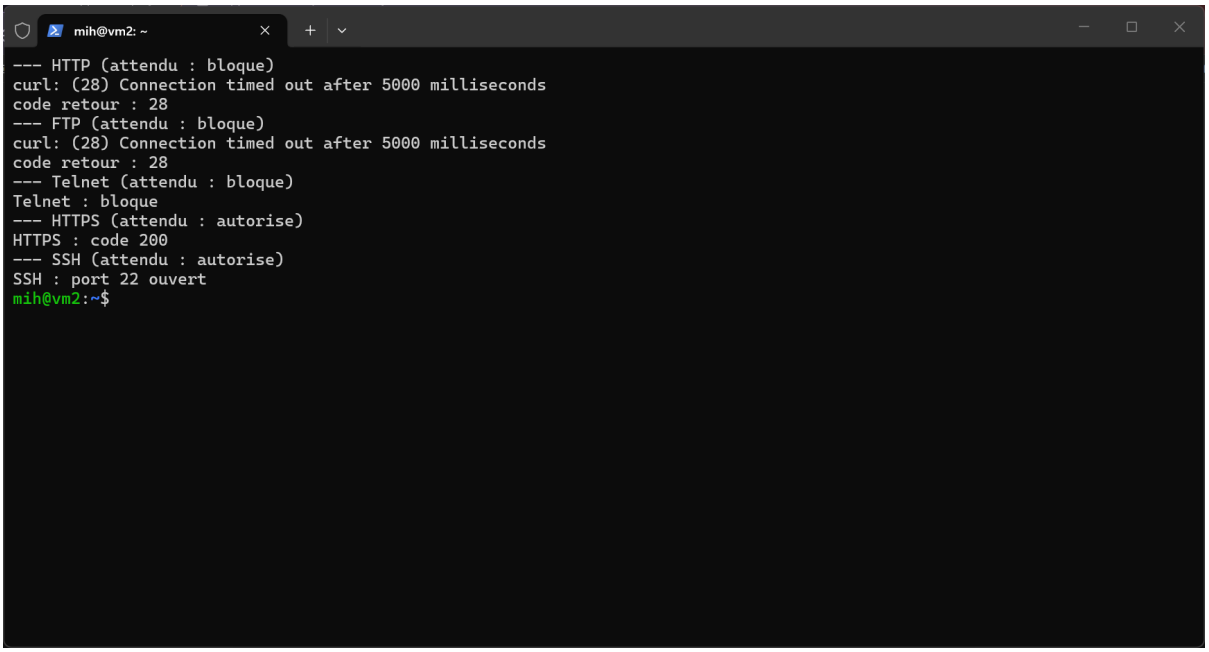


Figure 17 : depuis vm2, HTTP, FTP et Telnet sont bloqués ; HTTPS et SSH restent accessibles.

7.3 Journalisation

Le journal du noyau (`sudo dmesg | grep P01-BLOCK`) enregistre les connexions refusées : paquets `SYN` de 192.168.10.100 vers 192.168.20.10, avec `DPT=80` (HTTP), `DPT=21` (FTP) et `DPT=23` (Telnet). Les `SYN` rejetés sont envoyés à nouveau par le client à quelques secondes d'intervalle, ce qui explique les lignes répétées. Ce journal permet de détecter qu'un poste tente d'utiliser un protocole interdit.

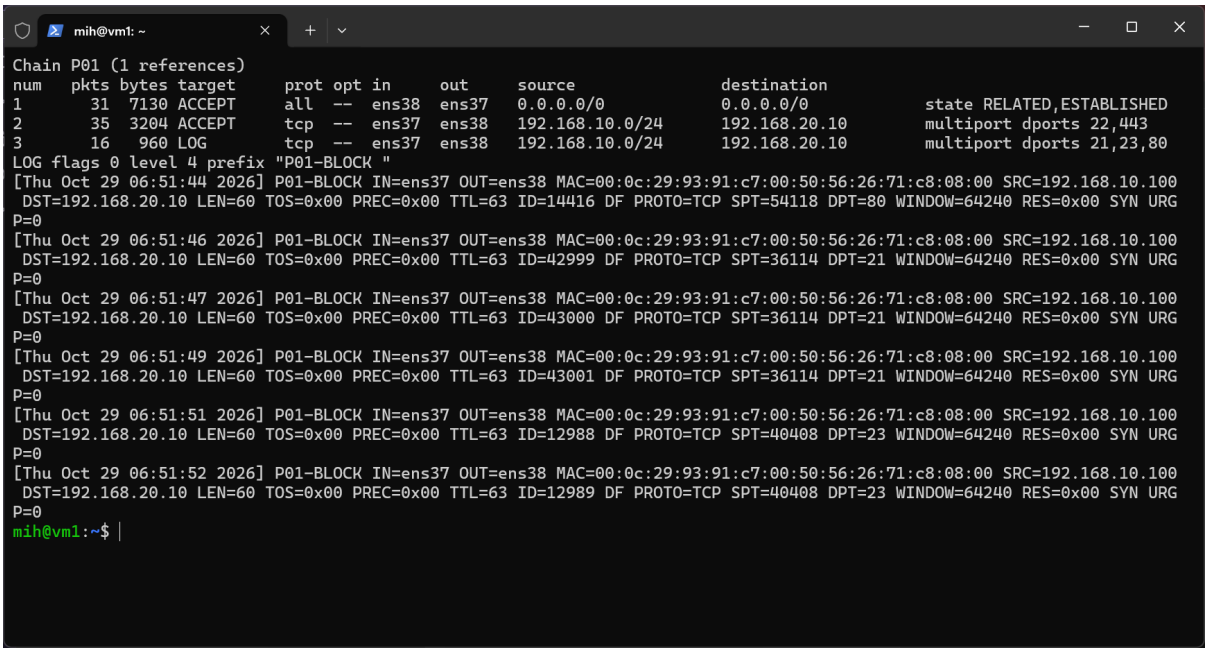


Figure 18 : sur vm1, la chaîne `P01` en phase défensive (règles 1 à 3) et les entrées `P01-BLOCK` du journal.

7.4 Portée de cette correction

Le pare-feu empêche l'usage des protocoles en clair entre les deux réseaux, mais vsftpd, telnetd et le port 80 d'Apache restent actifs sur vm3 : un attaquant déjà présent sur le LAN 2, ou sur vm3 elle-même, y aurait toujours accès. La correction complète consiste à désinstaller ou désactiver ces services, comme indiqué en section 9.

8. Analyse de sécurité

Les captures démontrent une atteinte à la **confidentialité** : un observateur du réseau obtient identifiants, fichiers et noms de serveurs, sans exploiter de vulnérabilité ni forcer de mot de passe. L'intégrité n'a pas été testée : sur un flux en clair, un attaquant actif pourrait aussi modifier les échanges.

8.1 Menaces, impacts et gravité

Actif exposé	Menace	Impact	Gravité
Identifiants HTTP, FTP, Telnet	écoute du trafic (technique ATT&CK « Network Sniffing »)	usurpation de compte, accès aux services	élevée
Fichiers transférés (FTP, Telnet)	écoute du trafic	consultation de données confidentielles	élevée
Requêtes DNS	écoute du trafic	cartographie des serveurs internes, reconnaissance	moyenne
Métadonnées du trafic chiffré (IP, SNI, tailles, horaires)	analyse de trafic	connaissance des services utilisés	faible à moyenne
Version des logiciels (bannière SSH)	lecture de la bannière	aide au choix d'une vulnérabilité connue	faible

La faiblesse commune est la transmission d'informations sensibles en clair (famille CWE-319).

8.2 Pourquoi l'attaque est simple

- **Aucune compétence cryptographique :** le « décodage » de l'authentification basique est un base64, et les mots de passe FTP et Telnet n'ont rien à décoder.
- **Outils gratuits et standard :** `tcpdump`, `tshark` ou Wireshark suffisent.
- **Position d'observation accessible :** routeur compromis, port miroir d'un commutateur, point d'accès Wi-Fi, ou poste ayant détourné le trafic.
- **Effet en cascade :** dans le laboratoire, le compte `labuser` sert à la fois pour FTP, Telnet et SSH. Le mot de passe volé sur un protocole en clair donne donc aussi accès à SSH, pourtant chiffré. La réutilisation d'un mot de passe annule le bénéfice du chiffrement.

8.3 Efficacité des contre-mesures

Aucune mesure ne suffit seule : le chiffrement protège le contenu, le pare-feu impose la politique, et la journalisation permet de repérer un poste qui s'écarte de cette politique.

Mesure	Ce qu'elle empêche	Ce qu'elle laisse
HTTPS (TLS 1.3)	lecture des identifiants et des pages	nom du serveur (SNI), adresses, tailles, horaires
SSH	lecture du mot de passe, des commandes et des fichiers	adresses, bannière, algorithmes, tailles
Pare-feu : seuls 22 et 443	usage de FTP, Telnet et HTTP entre les réseaux	services toujours actifs sur le serveur
Journalisation `P01-BLOCK`	rien directement	détecte les tentatives de protocoles interdits

9. Limites et recommandations

9.1 Limites du travail

- **Position de capture privilégiée :** la capture se fait sur le routeur vm1. Aucune attaque active (usurpation ARP, point d'accès pirate) n'a été menée pour se placer sur le chemin ; ces techniques relèvent d'autres sujets du catalogue.
- **Trafic de test :** le trafic vient de commandes (`curl`, `ssh`, `telnet`) lancées par le groupe, en faible volume, sans utilisateurs réels. Chaque scénario a été capturé une fois.
- **Vérification du certificat désactivée :** avec `curl -k`, le certificat auto-signé n'est pas contrôlé. Sans validation, un attaquant actif pourrait se placer entre le client et le serveur HTTPS.
- **Protocoles chiffrés non testés :** SFTP, FTPS et le DNS chiffré (DoT, DoH) n'ont pas été mis en place.
- **Comptes de test faibles et partagés :** `labuser` sert pour trois services, avec un mot de passe simple, volontairement.
- **Système en fin de support :** Ubuntu 20.04 a atteint la fin de son support standard le 31 mai 2025 (message affiché à la connexion). Ce choix, hérité du laboratoire INFO 351, ne convient pas à un système réel.
- **Règles non persistantes :** la chaîne `P01` disparaît au redémarrage de vm1, ce qui est voulu pour le laboratoire mais à corriger en production.

9.2 Recommandations

Les priorités 1 à 3 règlent l'essentiel du risque démontré ; les suivantes réduisent l'impact d'une compromission et la surface d'attaque.

Priorité	Recommandation	Risque traité
1	Désinstaller `vsftpd` et `telnetd`, et fermer ou rediriger le port 80	protocoles en clair encore actifs sur le serveur
2	Remplacer FTP par SFTP (ou FTPS) et Telnet par SSH	fuite des identifiants et des fichiers
3	Imposer HTTPS : redirection 80 vers 443, HSTS, TLS 1.2 et 1.3 seulement, certificat d'une autorité de confiance (par exemple une PKI interne)	écoute et interception web
4	SSH par clés, mots de passe désactivés	vol ou devinette de mot de passe
5	Un mot de passe distinct par service, avec authentification multifacteur quand c'est possible	effet en cascade d'un mot de passe volé
6	Chiffrer le DNS des postes (DoT ou DoH)	cartographie du réseau par l'écoute
7	Protéger la couche 2 (ports miroirs contrôlés, 802.1X, inspection ARP, DHCP snooping)	placement d'un attaquant sur le chemin
8	Envoyer les journaux `P01-BLOCK` vers un outil de supervision et alerter sur les ports interdits	détection des postes qui tentent un protocole interdit
9	Mettre à jour le système vers une version supportée	vulnérabilités connues

10. Répartition des rôles et conclusion

10.1 Rôle de chaque membre

Membre	Rôle	Parties du rapport
Miriah	Architecture, installation de vm3, règles de vm1	2, 3, 4
Joe	Captures et analyse HTTP et FTP	5.1, 5.2
Soatiana	Captures et analyse Telnet et DNS, mise en forme Wireshark	5.3, 5.4
Laurie	HTTPS, SSH, phase défensive, analyse de sécurité	6, 7, 8, 9

Livrables associés. Dépôt Git : https://github.com/Miriah-santatriniaina/p01-capture-analyse-trafic (scripts `p01_vm3_install.sh` et `p01_vm1_flux.sh`, six fichiers `.pcap`, captures d'écran).

10.2 Conclusion

Le projet répond aux deux volets demandés. En phase offensive, une simple écoute du trafic entre un client et un serveur suffit pour lire en clair le mot de passe HTTP (`webadmin:Secret2026`), les identifiants et le fichier transférés en FTP, toute une session Telnet et les noms résolus par le DNS. En phase de comparaison, HTTPS (TLS 1.3, `TLS_AES_256_GCM_SHA384`) et SSH (`chacha20-poly1305`) rendent les mêmes échanges illisibles : plus aucune occurrence des mots de passe dans les captures.

La phase défensive ferme les flux en clair au niveau du routeur et le prouve par des tests de blocage et un journal. Elle ne remplace pas la suppression des services non sécurisés, ni la protection des métadonnées (nom du serveur, adresses, tailles), qui restent visibles même chiffrées. L'enseignement principal est qu'il faut chiffrer par défaut, supprimer ce qui ne l'est pas, et utiliser un mot de passe différent pour chaque service.

Annexes

A. Reproduction pas à pas

État de départ : les trois machines du laboratoire INFO 351 (vm1, vm2, vm3), avec accès à Internet par vm1 pour installer les paquets.

1. Sur vm3 : installer les services avec le script.

```
sudo bash ~/p01_vm3_install.sh
```

2. Sur vm2 : installer les clients.

```
sudo apt update && sudo apt install -y ftp telnet curl
```

3. Sur vm1 : ouvrir les flux, puis capturer (un nom de fichier différent par capture).

```
sudo bash ~/p01_vm1_flux.sh offensif
sudo bash ~/p01_vm1_flux.sh capture clair_http "host 192.168.20.10 and port 80"
```

4. Sur vm2, pendant la capture : générer le trafic, puis arrêter la capture avec Ctrl+C sur vm1.

```
curl -u webadmin:Secret2026 http://web.esti.mg/prive/
curl -u labuser:Lab@2026 ftp://web.esti.mg/confidentiel.txt
telnet web.esti.mg
curl -k -u webadmin:Secret2026 https://web.esti.mg/prive/
ssh labuser@web.esti.mg 'whoami; cat confidentiel.txt'
```

5. Sur vm1 : extraire les informations des captures.

```
sudo tshark -r /root/captures/clair_http.pcap -Y http.authbasic -T fields -e ip.src -e
http.authbasic
sudo tshark -r /root/captures/clair_ftp.pcap -Y 'ftp.request.command=="USER" ||
ftp.request.command=="PASS"' \
  -T fields -e ip.src -e ftp.request.command -e ftp.request.arg
sudo tshark -r /root/captures/clair_telnet.pcap -q -z follow,tcp,ascii,0
sudo grep -a -c Secret2026 /root/captures/chiffre_https.pcap
sudo grep -a -c -e Lab@2026 -e confidentiel /root/captures/chiffre_ssh.pcap
```

6. Sur vm1 : appliquer la correction, puis revenir à l'état initial si besoin.

```
sudo bash ~/p01_vm1_flux.sh defensif
sudo bash ~/p01_vm1_flux.sh reset
```

B. Script de gestion des flux et de la capture (vm1)

```
#!/bin/bash
# P01 - gestion des flux LAN1 -> vm3 et capture (a executer en root sur vm1)
#   sudo bash p01_vm1_flux.sh offensif | defensif | reset | capture NOM [filtre]

VM3="192.168.20.10"
LAN1="192.168.10.0/24"
IF_LAN1="ens37"
IF_LAN2="ens38"
CHAIN="P01"

init_chain() {
  iptables -N $CHAIN 2>/dev/null || true
  iptables -F $CHAIN
```

```
  iptables -C FORWARD -j $CHAIN 2>/dev/null || iptables -I FORWARD 1 -j $CHAIN
  iptables -A $CHAIN -i $IF_LAN2 -o $IF_LAN1 -m state --state ESTABLISHED,RELATED -j ACCEPT
}

case "$1" in
  offensif)
    init_chain
    iptables -A $CHAIN -i $IF_LAN1 -o $IF_LAN2 -s $LAN1 -d $VM3 -p tcp \
      -m multiport --dports 21,22,23,80,443,40000:40010 -j ACCEPT
    echo "Phase offensive : flux en clair ET chiffres ouverts vers $VM3"
    ;;
  defensif)
    init_chain
    iptables -A $CHAIN -i $IF_LAN1 -o $IF_LAN2 -s $LAN1 -d $VM3 -p tcp \
      -m multiport --dports 22,443 -j ACCEPT
    iptables -A $CHAIN -i $IF_LAN1 -o $IF_LAN2 -s $LAN1 -d $VM3 -p tcp \
      -m multiport --dports 21,23,80 -j LOG --log-prefix "P01-BLOCK " --log-level 4
    echo "Phase defensive : seuls SSH (22) et HTTPS (443) passent vers $VM3"
    ;;
  reset)
    iptables -D FORWARD -j $CHAIN 2>/dev/null || true
    iptables -F $CHAIN 2>/dev/null || true
    iptables -X $CHAIN 2>/dev/null || true
    echo "Regles P01 supprimees"
    ;;
  capture)
    NOM="${2:?Donnez un nom de capture, ex: clair_http}"
    FILTRE="${3:-host $VM3}"
    mkdir -p ~/captures
    echo "Capture sur $IF_LAN2 -> ~/captures/$NOM.pcap (Ctrl+C pour arreter)"
    tcpdump -i $IF_LAN2 -nn -s 0 -w ~/captures/$NOM.pcap $FILTRE
    ;;
  *)
    echo "Usage : $0 {offensif|defensif|reset|capture NOM [filtre]}"
    exit 1
    ;;
esac

iptables -L P01 -n -v --line-numbers 2>/dev/null
```

C. Configurations principales du serveur (vm3)

Extraits du script `p01_vm3_install.sh`, fourni en entier dans le dépôt Git.

```
# /etc/vsftpd.conf (FTP en clair, ports passifs 40000-40010)
listen=YES
anonymous_enable=NO
local_enable=YES
write_enable=YES
pam_service_name=vsftpd
pasv_enable=YES
pasv_min_port=40000
pasv_max_port=40010

# /etc/apache2/conf-available/p01-auth.conf (authentification basique)
<Directory /var/www/html/prive>
    AllowOverride None
    AuthType Basic
    AuthName "Zone privee P01"
```

```
    AuthUserFile /etc/apache2/.htpasswd-p01
    Require valid-user
</Directory>

# /etc/apache2/sites-available/p01-ssl.conf (HTTPS)
<VirtualHost *:443>
    ServerName web.esti.mg
    DocumentRoot /var/www/html
    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/p01.crt
    SSLCertificateKeyFile /etc/ssl/private/p01.key
</VirtualHost>
```

D. Références

- RFC 854 (Telnet), RFC 959 (FTP), RFC 1035 (DNS), RFC 7617 (authentification HTTP « Basic »), RFC 8446 (TLS 1.3), RFC 4253 (couche transport SSH), RFC 7858 (DNS sur TLS), RFC 8484 (DNS sur HTTPS).
- MITRE ATT&CK, technique T1040 « Network Sniffing ».
- CWE-319 « Cleartext Transmission of Sensitive Information ».